GOSIM

GOSIM Shenzhen 2026

趋境科技 APPROACHING.AI

KTransformers:

The Fine-Tuning-to-Inference Closed Loop for Full-Precision 1T-Class MoE Models on Consumer Hardware

Peilin Li, Yujie Yang @ KVCache.AI

https://github.com/kvcache-ai/ktransformers



4.0-16.1x
throughput improvement
over ZeRO-Offload

128K
long context
without OOM

single 4090
same as inference; DS-671B BF16 case

Content

01 KTransformers Inference

Why CPU Co-Compute Replaces Offload

02 KTransformers Fine-Tuning

Layout and Scheduling Under Routed Experts
Using Residual VRAM for LoRA Expert

03 Deployment And Ecosystem

Use Cases, Training-Inference Loop, And Research

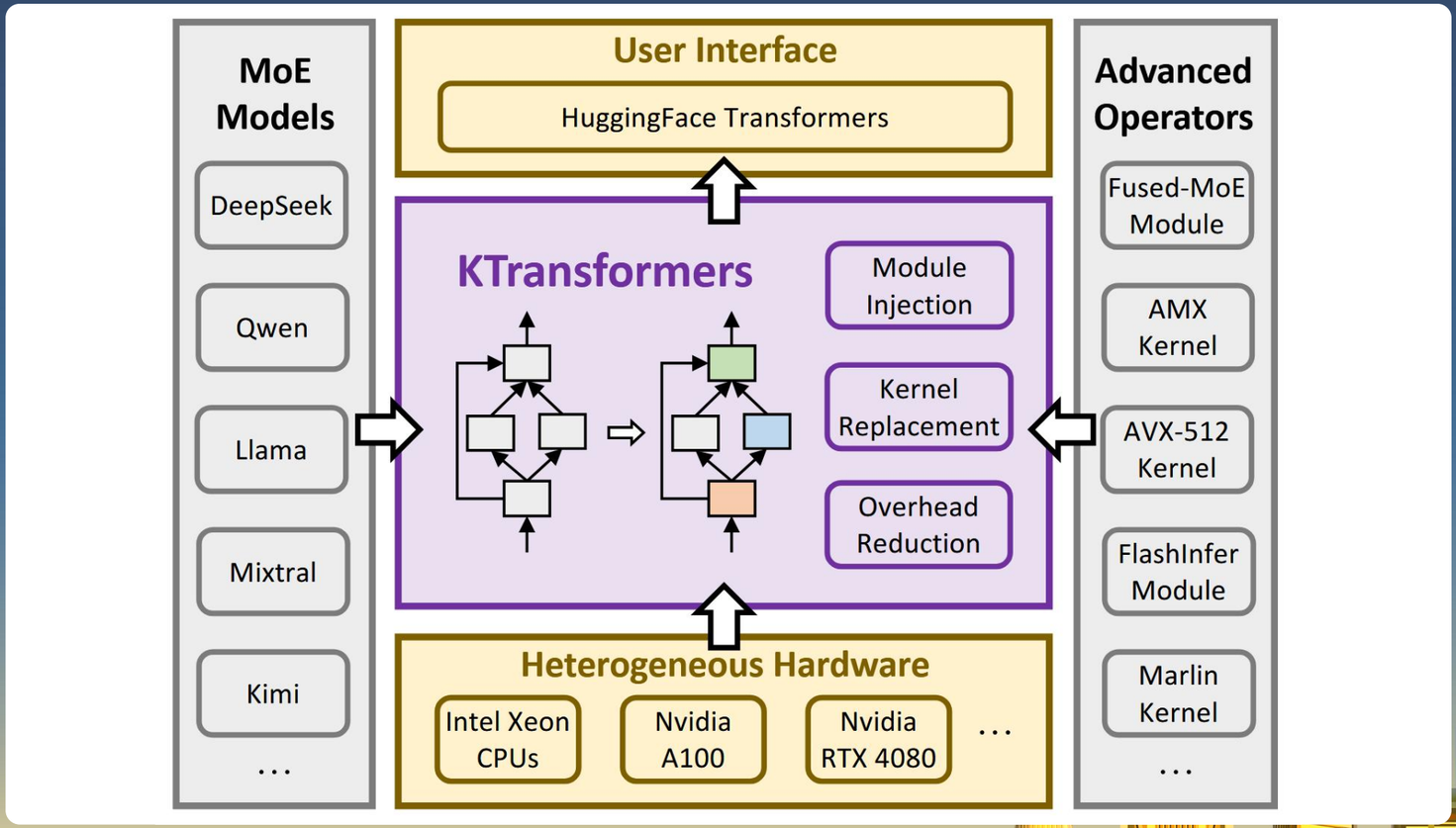
04 Four Advanced Features

Reporter: Yujie Yang

01

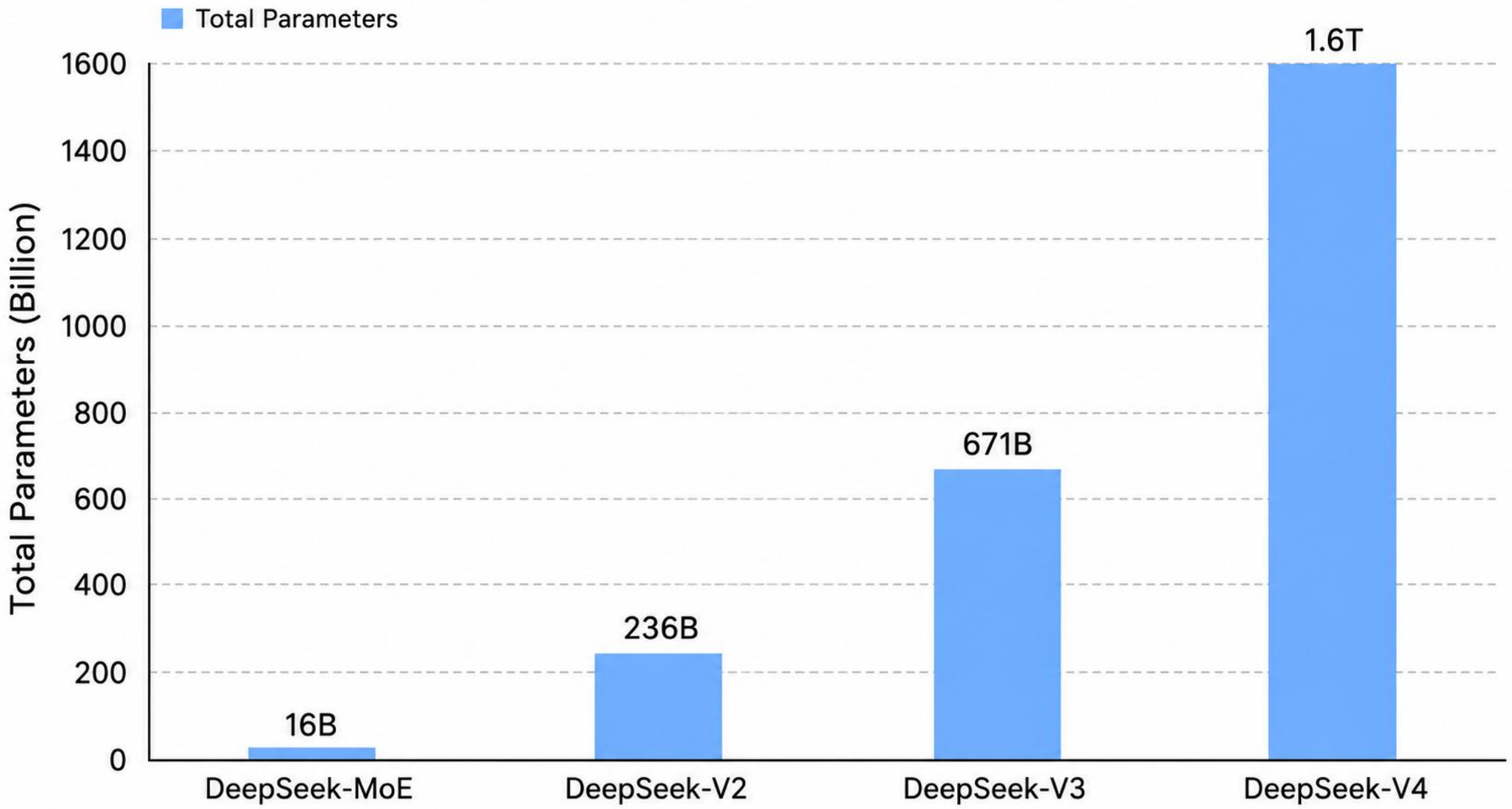
KTransformers Inference

Why CPU Co-Compute Replaces Offload

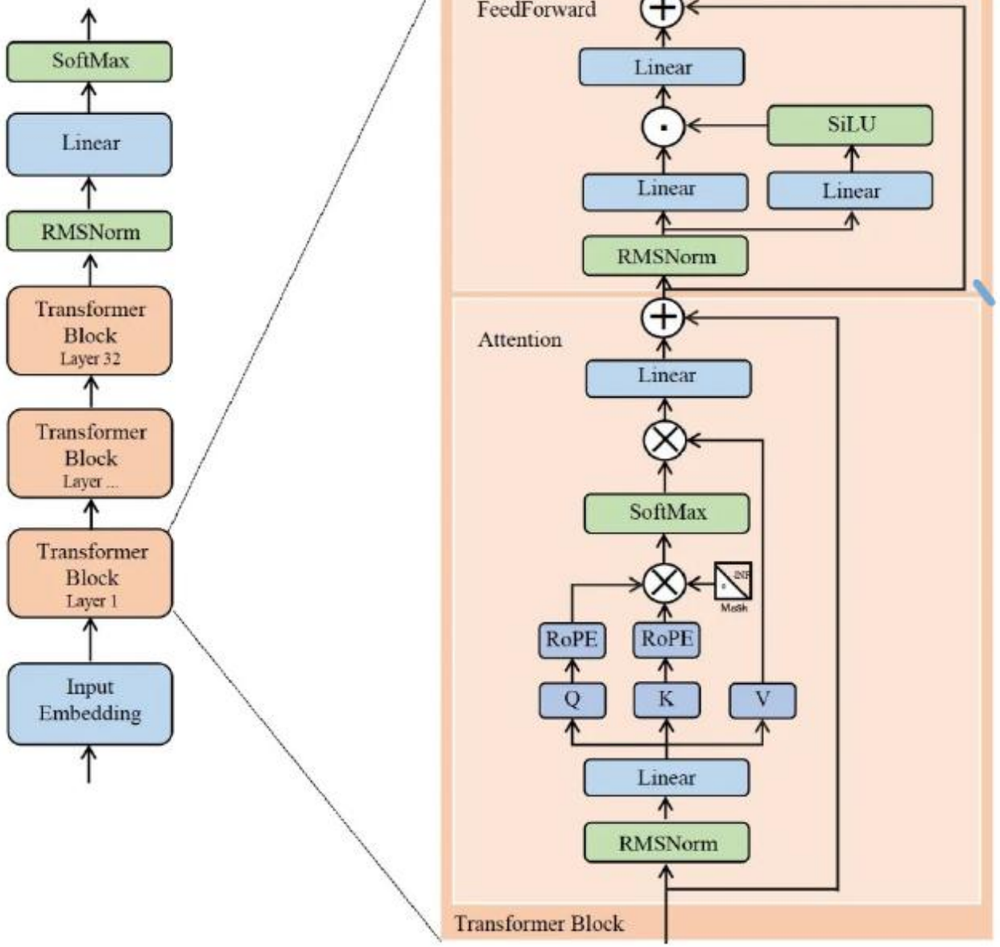


MoE Becomes Mainstream As Models Scale

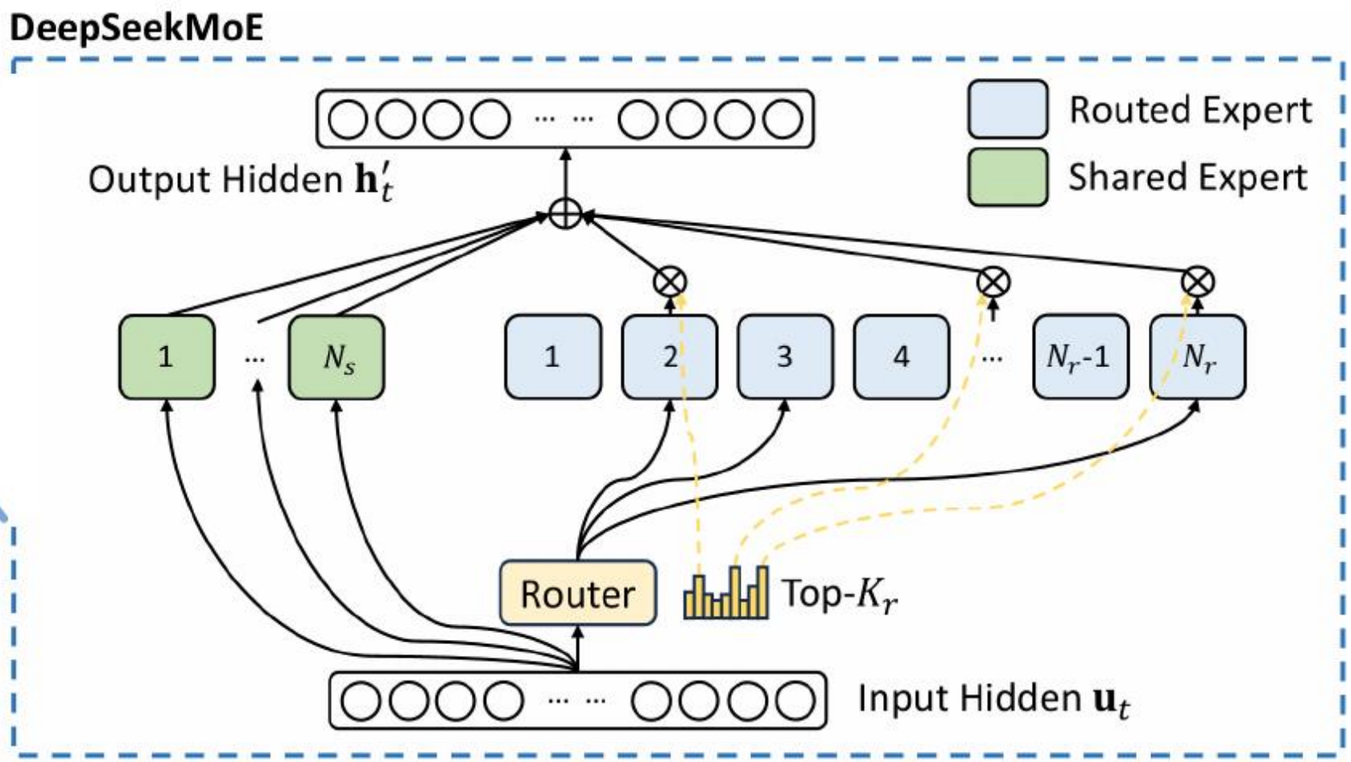
Recent MoE releases move from 16B-class systems to 1.6T-class frontier models.



Background: Sparse Mixture-of-Experts (MoE)



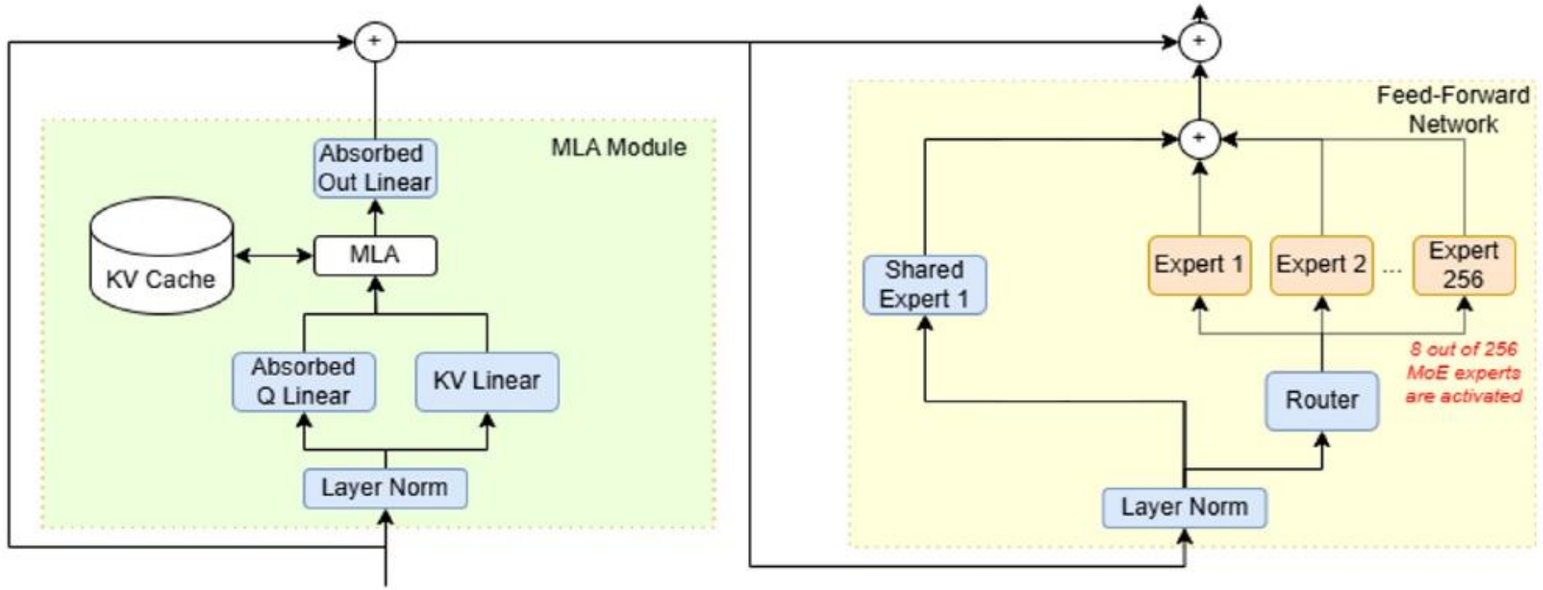
Transformer Architecture



Dense FFN → Sparse Mixture-of-Experts (MoE)

One large matrix ⇨ Hundreds of small matrices + Activate only 6~8 at a time

KTransformers: Arithmetic Intensity-Aware Offloading Strategy



Operator	MLA Attention	Norm & Linear & Shared Experts	Routed Experts
Total Size	~ 5B for 128K Context	~17B	~654B
Arithmetic Intensity	High	Medium	Low
	On a Single GPU		Offloaded to CPUs

Offload Priority:

Routed Experts

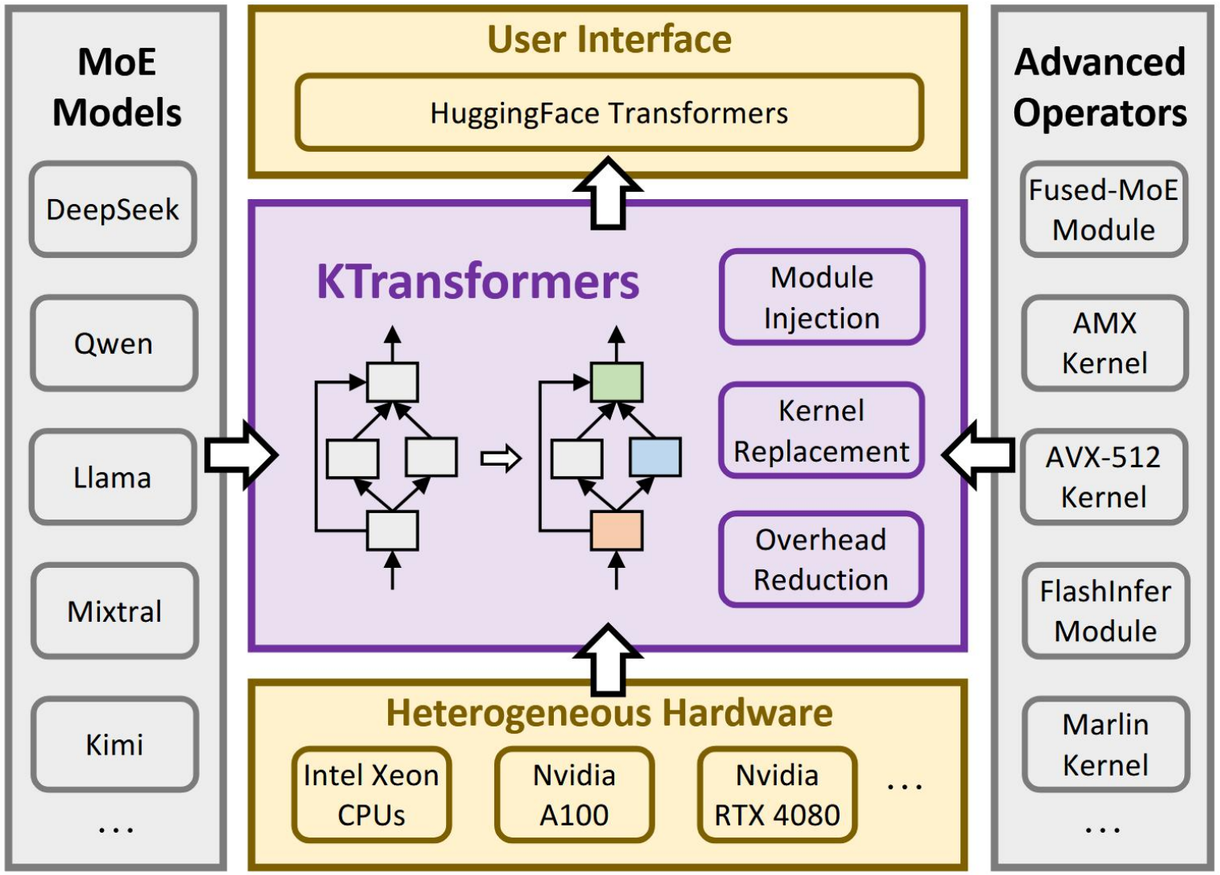
>

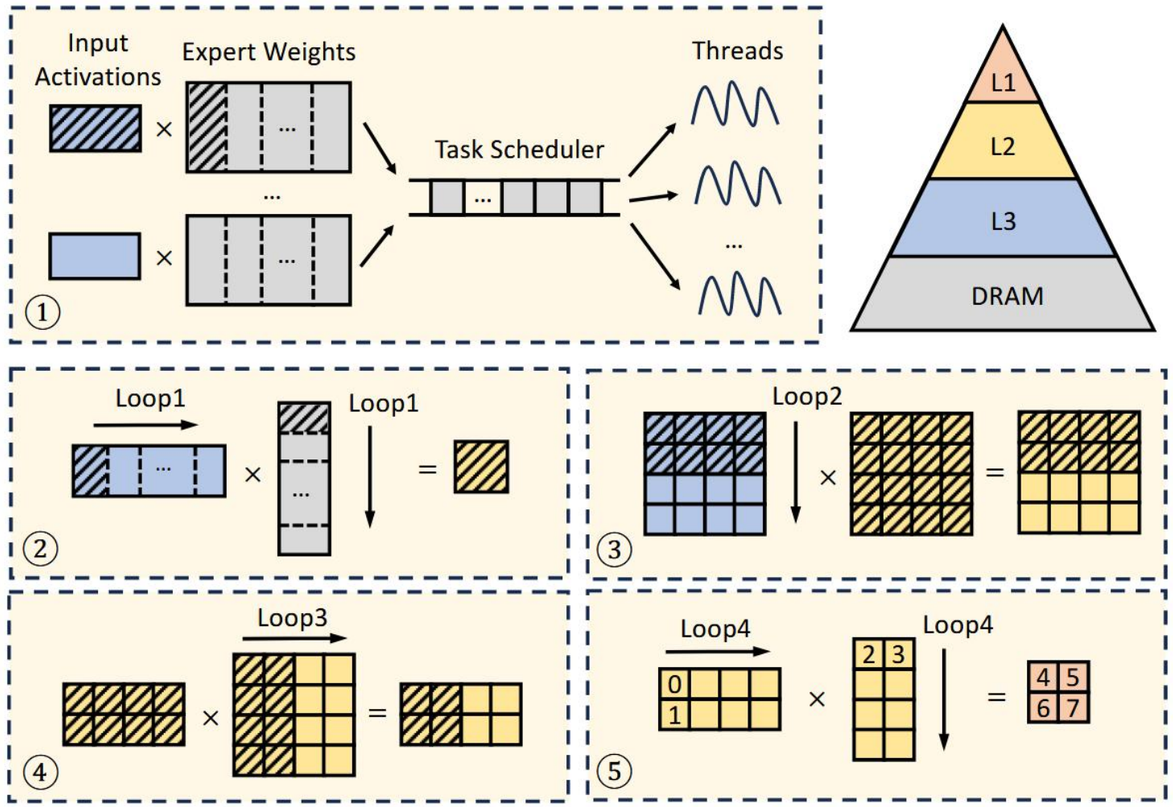
Shared Experts

>

MLA Attention

Core Technologies of KTransformers



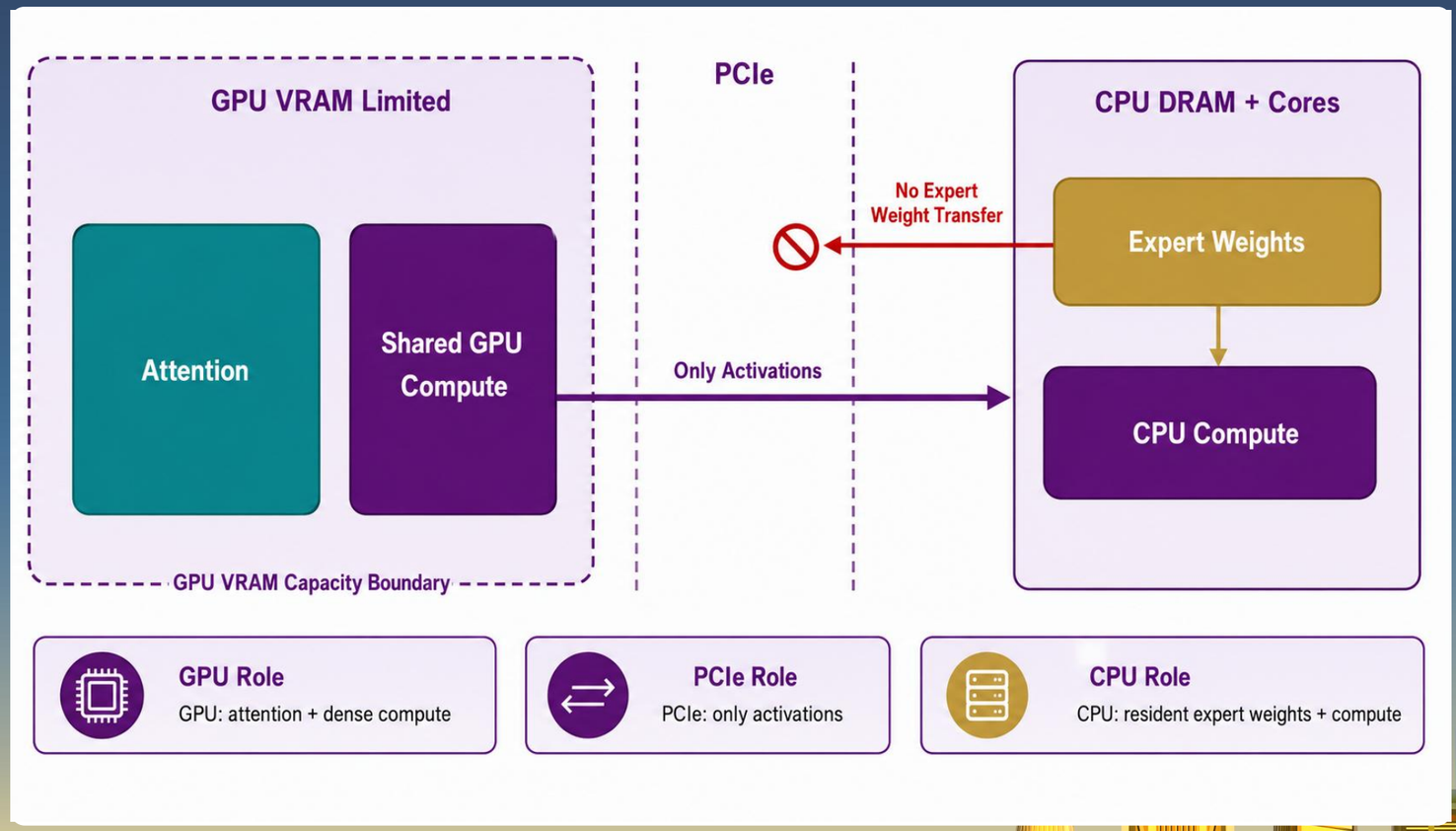


Overall KT-System and Optimize in Prefill & Decode

02

KTransformers Fine-Tuning

CPU-resident experts change what must cross PCIe.



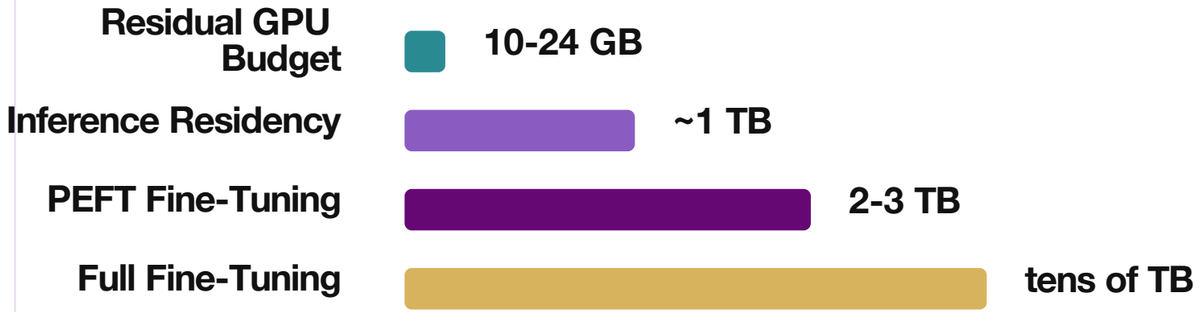
Fine-Tuning Turns Scale Into VRAM Demand

Full Fine-tuning adds training state on top of model weights, turning scale into a GPU-memory barrier.

Training State Adds Residency

Model Weights	large resident parameter pool
Activations	persist across forward and backward
Gradients	training adds reverse-path state
Optimizer State	adapters still require residency

PEFT Still Needs Multi-TB GPU Memory



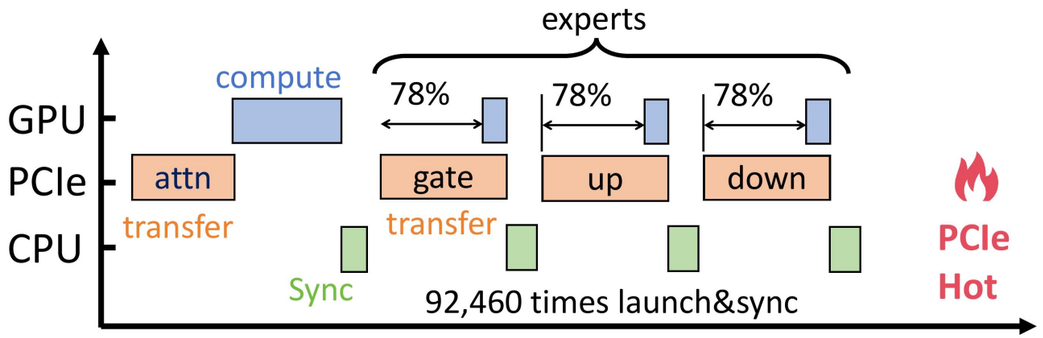
18xInference
full fine-tuning memory inflation

2-3 TB
PEFT GPU memory on frontier MoE

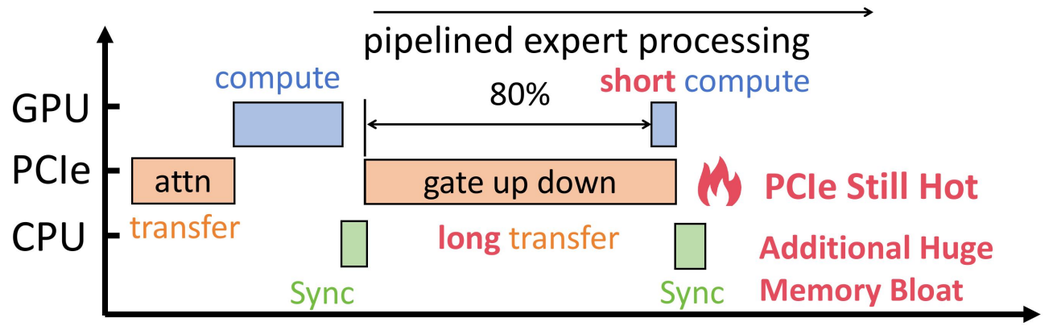
50+ A100
a major barrier for researchers and small companies

PEFT reduces trainable parameters, but multi-TB residency remains.

PCIe Should Carry Activations, Not Expert Weights



(b) Traditional Offload Paradigm (ZeRO-Offload)



(c) MoE Offload Paradigm (APTMoE)



(d) Co-compute Paradigm (Ours KT-FT)

PCIe works only when it carries activations, not resident expert weights.

Challenge 1

Transferring weights takes most step time.

78-80% PCIe time and up to 1.3 TB transfer per step.

Challenge 2

Small all-gathers increase latency.

Gate / up / down weights can trigger 122 calls per step.

Challenge 3

Prefetching can increase memory use.

PCIe stays busy, but host buffers grow with the stream.

Solution

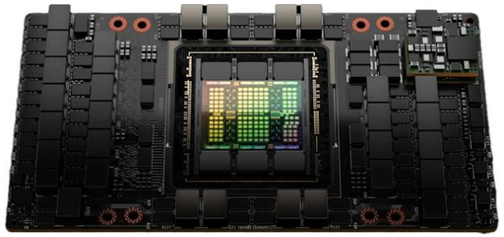
Compute where expert weights reside.

KT-FT runs routed expert work on CPU instead of transferring weights.

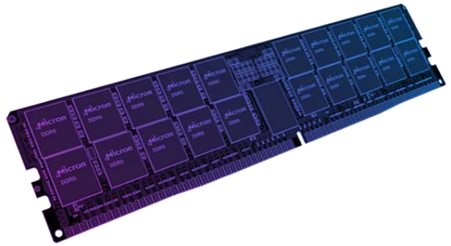
Attention Fits GPUs; Experts Fit CPU Memory

Hardware Resources

A100 GPU



Xeon SPR + DDR5



80GB VRAM ~2 TB/s bandwidth	8*64GB DRAM 8*40GB/s bandwidth
High peak bandwidth, expensive residency	Cheaper bandwidth, larger DRAM capacity

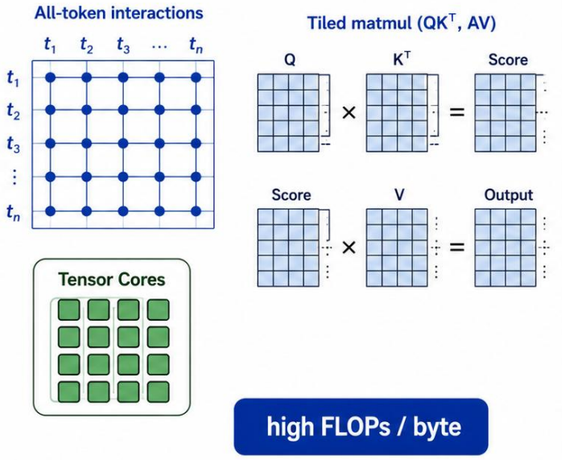
Model Bottlenecks Match Hardware

Compute-Bound

Memory-Bound

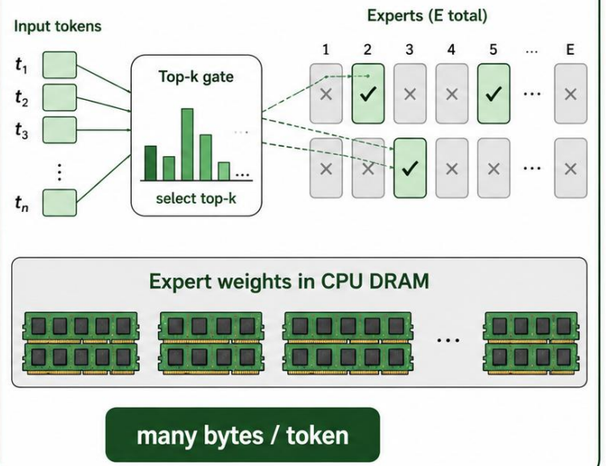
Dense Attention

GPU compute-bound

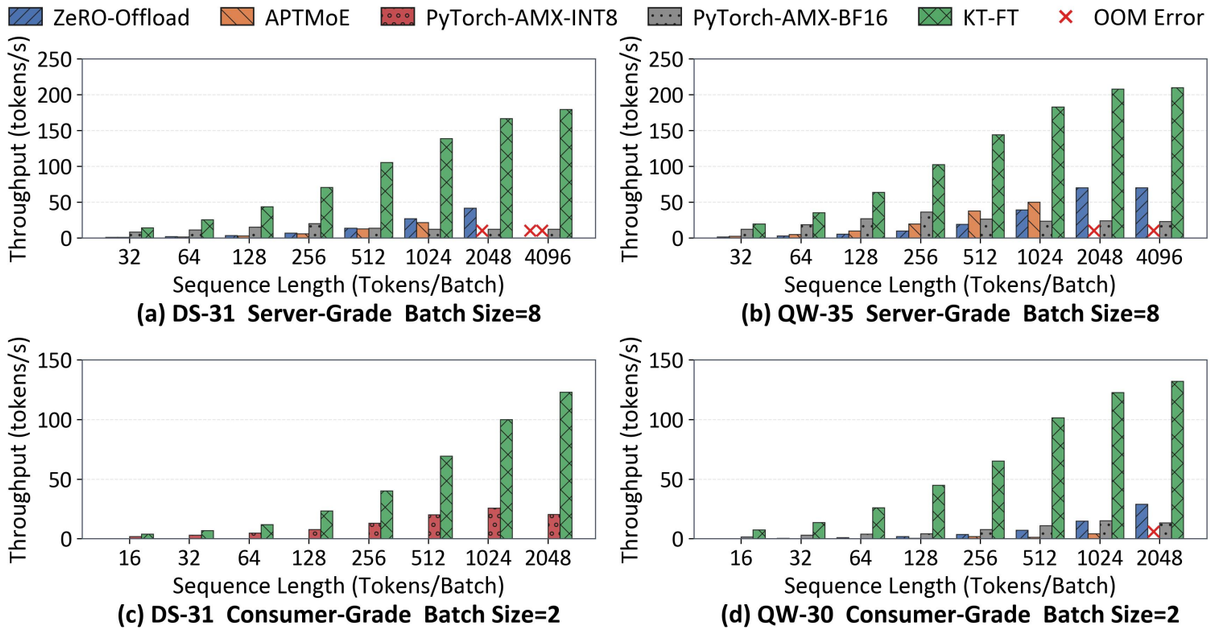


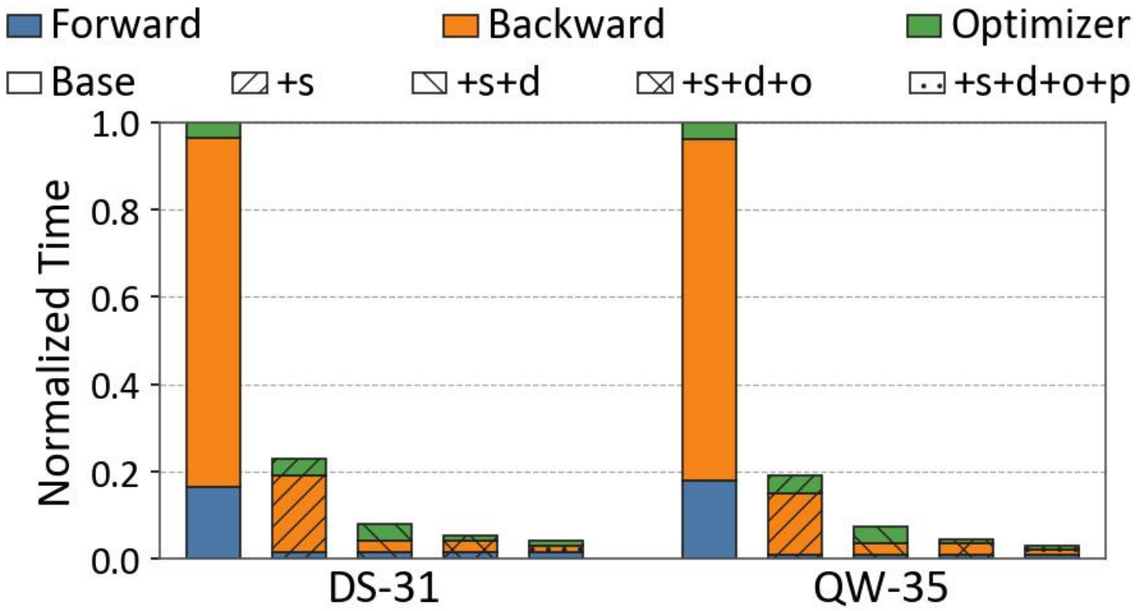
Sparse MoE Experts

CPU memory-bound



Co-Compute Raises Throughput By 4.0-16.1x





The end-to-end gains start with resident experts, then remove host-side layout and scheduling costs.

Paradigm **4.0-16.1x**

avoid transferring expert weights across PCIe

Layout **4.4x**

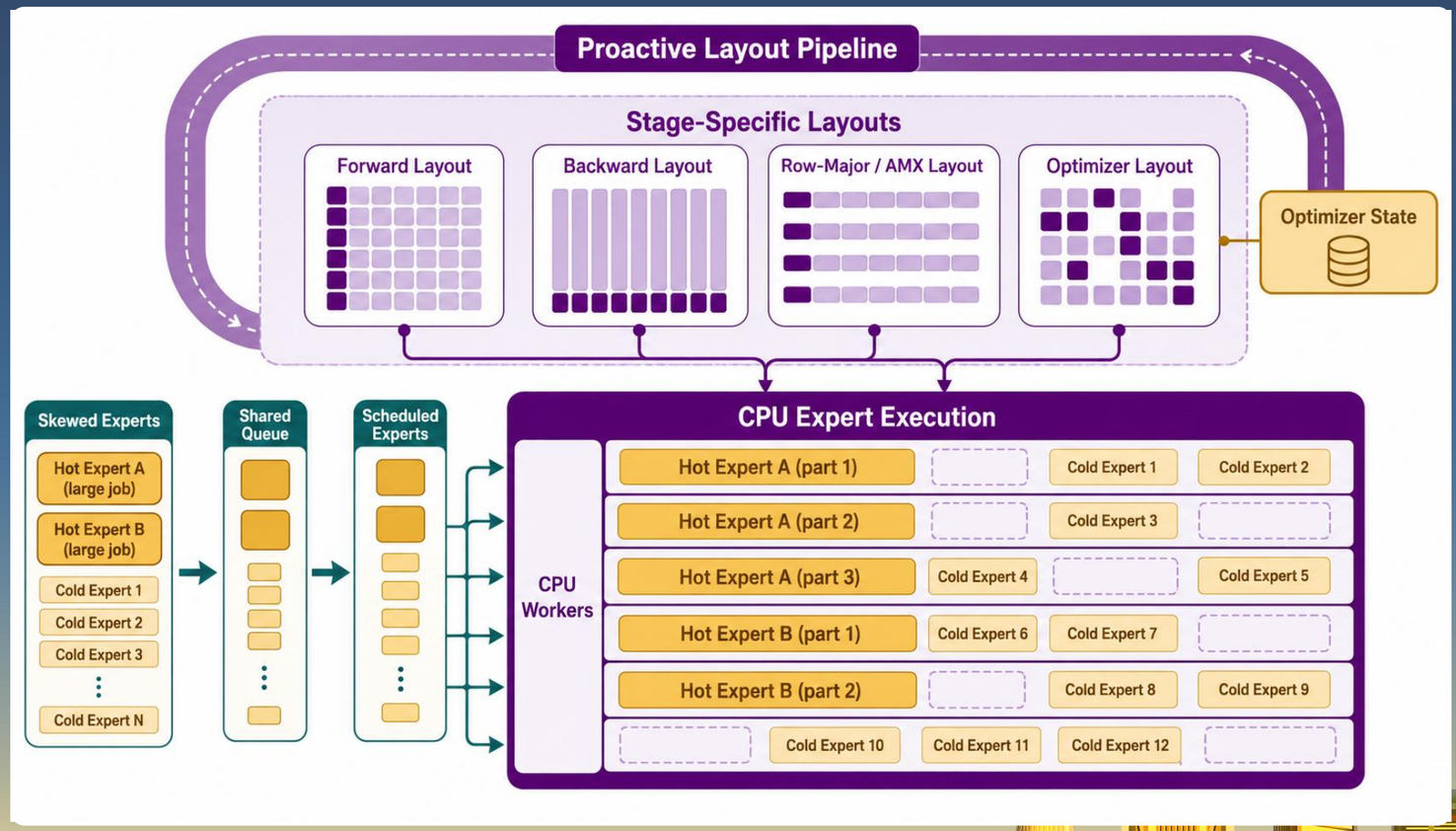
remove cross-stage data conversion

Scheduling **6.4x**

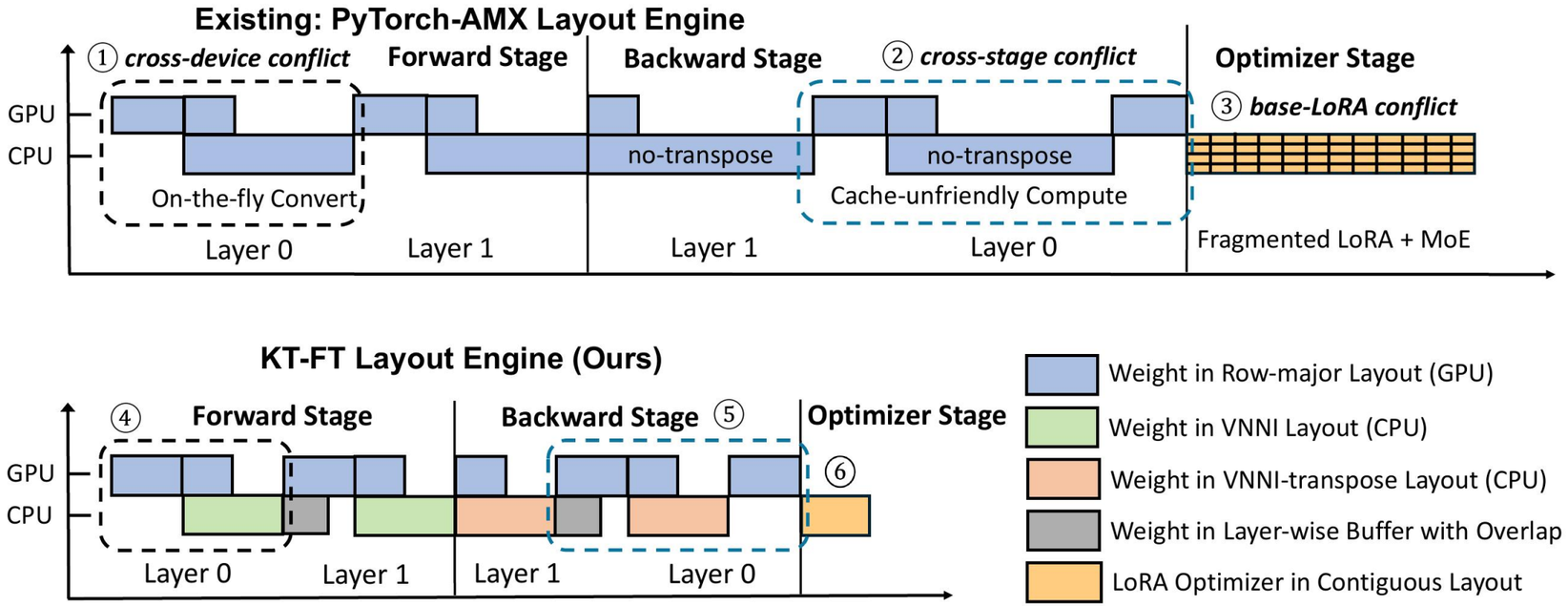
reduce routed-expert stalls in backward

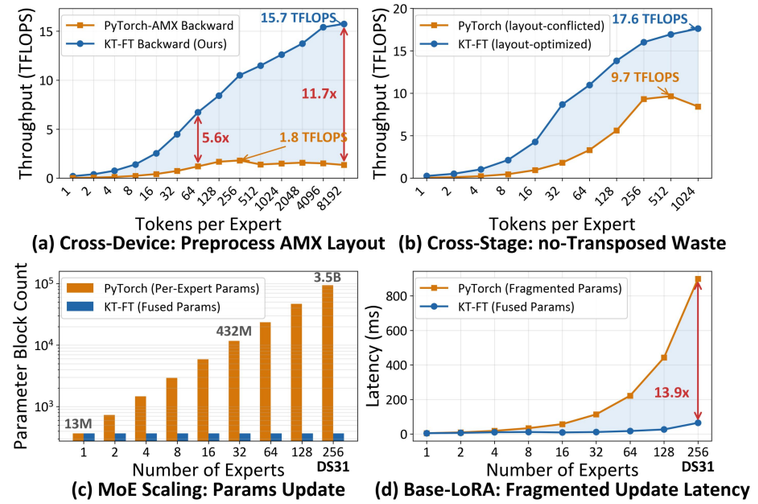
System Challenges And Algorithm Co-Design

Once placement changes, layout and scheduling become the next bottlenecks.



Co-Compute Creates Weight Layout Conflicts





Overhead Profile

Cross-Device Conflict

GPU row-major weights and CPU VNNI layouts fight each other.

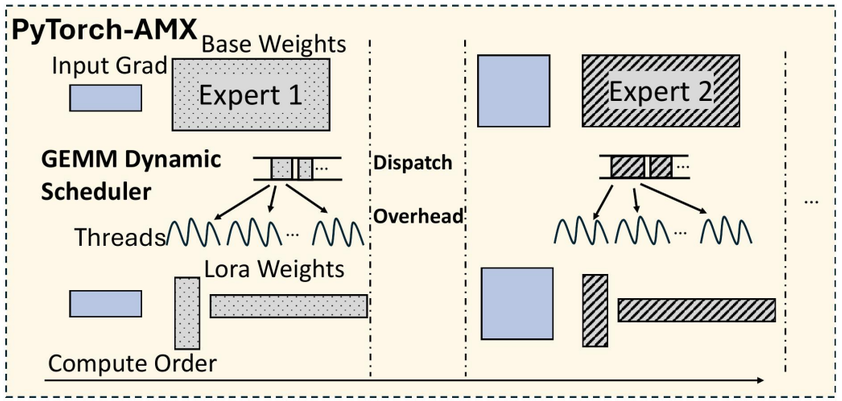
Cross-Stage Conflict

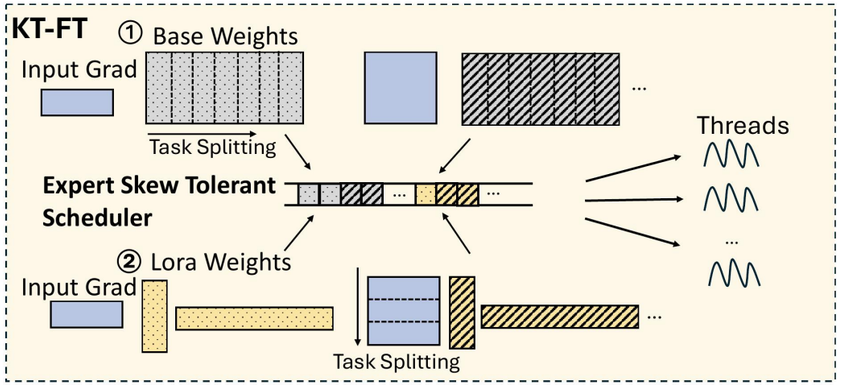
Forward, backward, and optimizer stages want different memory views.

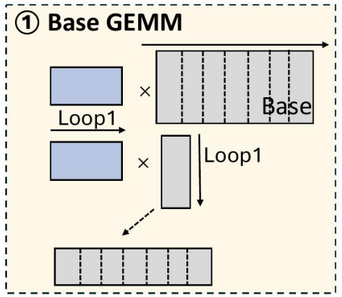
Base-LoRA Conflict

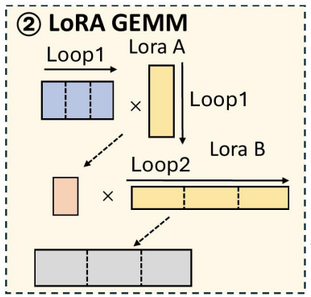
Fragmented LoRA updates create a small-write optimizer path.

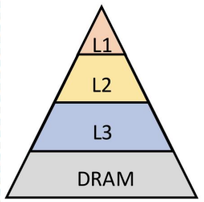
A Shared Queue Makes Routed Experts Skew-Tolerant











Task Decomposition Table

Table 1. Representative task decomposition across forward and backward phases. $|\mathcal{A}|$ denotes the number of active experts; P is the per-expert tile count determined by the AMX kernel; Q is the token count; N_t denotes the LoRA sequence-dimension parallelism; B_t is the token block size; Q_i is the token count of the i-th active expert.

Direction	Phase	Dimension	Task Count
Forward	Input gather	token	Q
	Input quantize	expert	$\lvert\mathcal{A}\rvert$
	Gate+Up GEMM	expert×tile	$\lvert\mathcal{A}\rvert \times P \times 2$
	Gate+Up LoRA	expert×token	$\lvert\mathcal{A}\rvert \times N_t \times 2$
	Down GEMM	expert×tile	$\lvert\mathcal{A}\rvert \times P$
	Output merge	token	Q
Backward	Grad scatter	token	Q
	Grad intermediate	expert×tile	$\lvert\mathcal{A}\rvert \times P$
	LoRA grad project	expert×token	$\sum_i \lceil Q_i / B_t \rceil$
	Activation backward	expert	$\lvert\mathcal{A}\rvert$
	Grad input	expert×tile	$\lvert\mathcal{A}\rvert \times P \times 2$
	LoRA grad input	expert×token	$\lvert\mathcal{A}\rvert \times N_t \times 2$
	Grad accumulate	token	Q

0 Baseline Problem

each expert GEMM is one job; threads split inside it

1 Shared Runtime Queue

mix hot and cold expert tasks across CPU workers

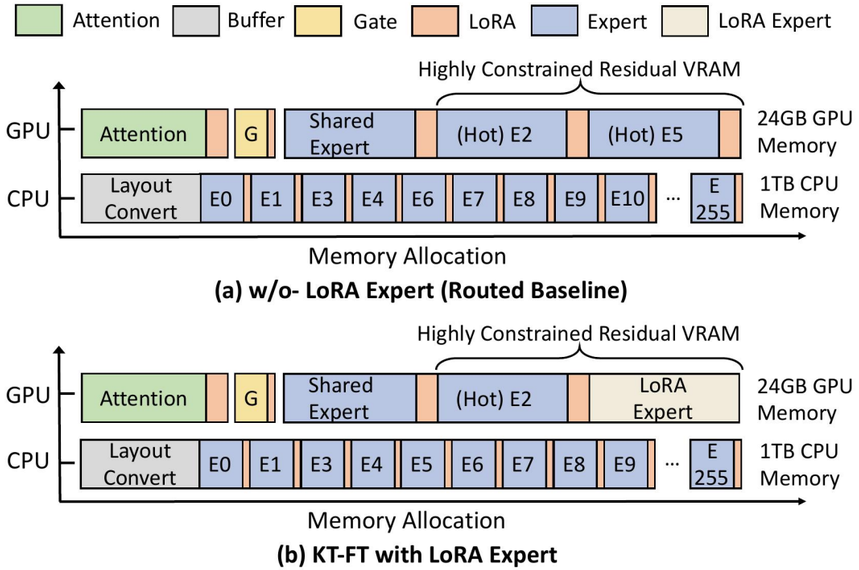
2 Task Decomposition

split Base and LoRA tasks; separate phase and dimension

Shared queue plus task decomposition makes dispatch skew-tolerant.

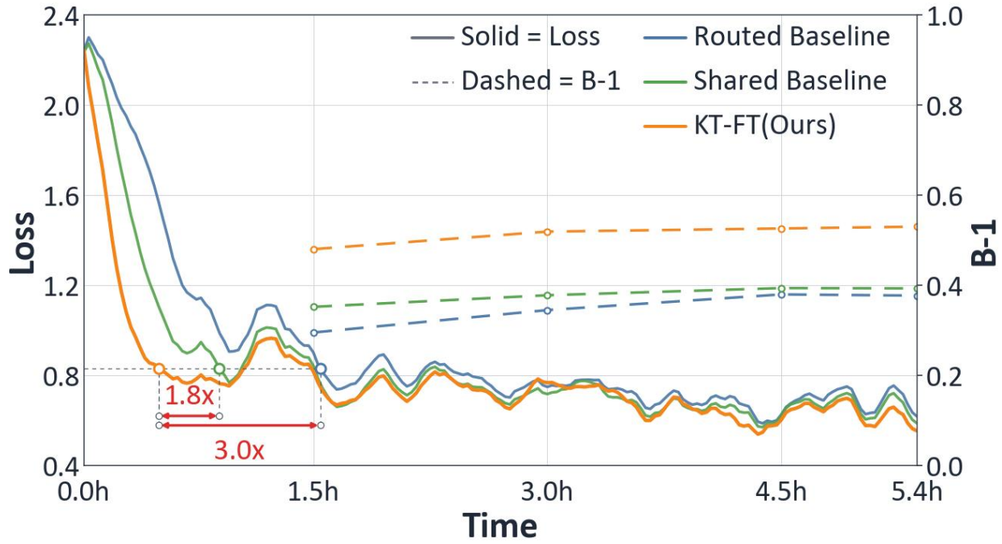
LoRA Expert Improves Time-To-Quality

Use residual VRAM for shared adaptation, not caches.



- Static and GPU-resident.
- Routing-independent dense gradient path.
- Fits into the remaining GPU compute bubble.

Shared GPU adaptation improves quality while CPU experts keep memory resident.



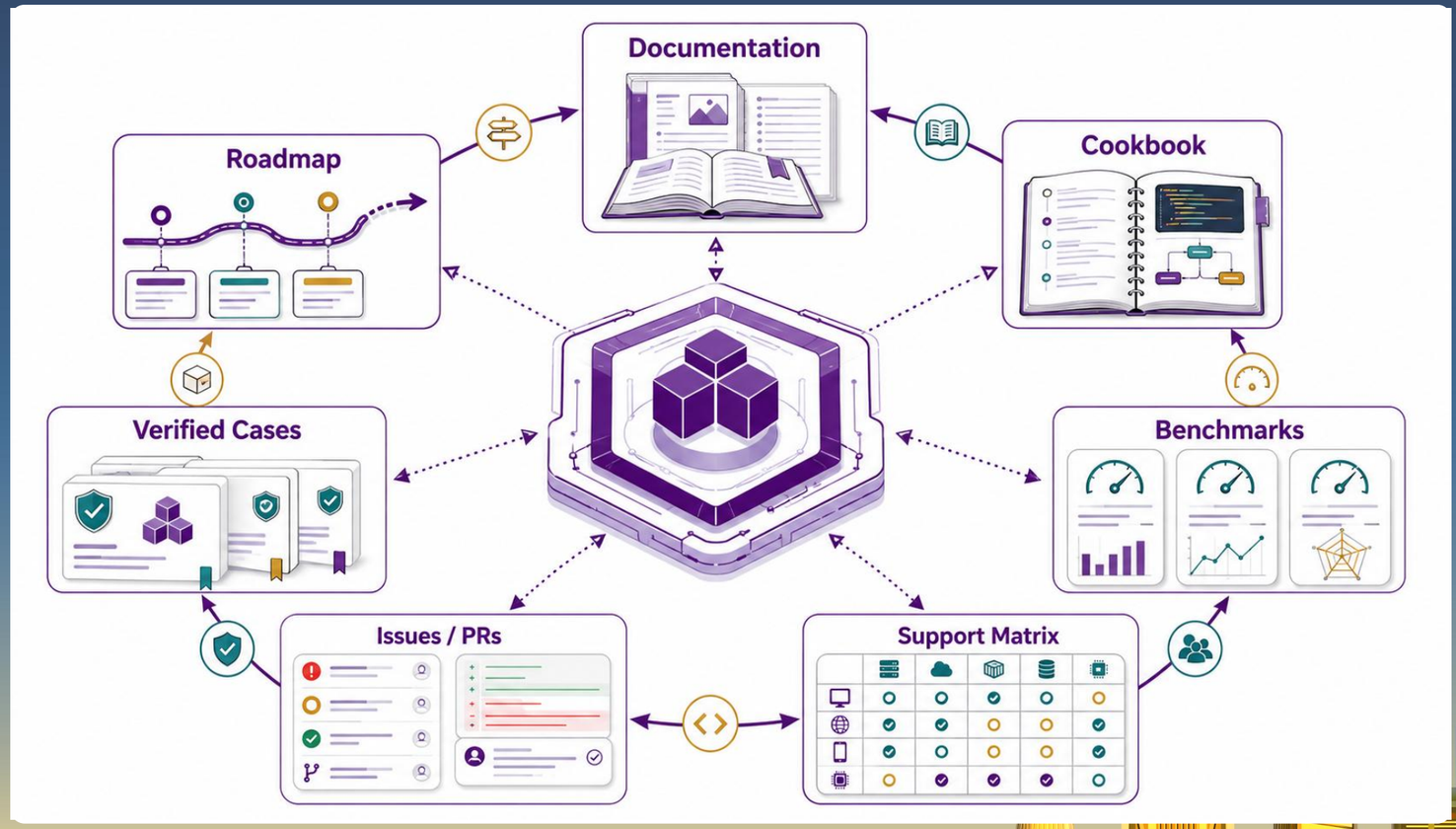
System headroom becomes an algorithmic opportunity.

1.8-3.0x	3.8-4.2%	1.2GB
TTQ acceleration	score gains	residual VRAM

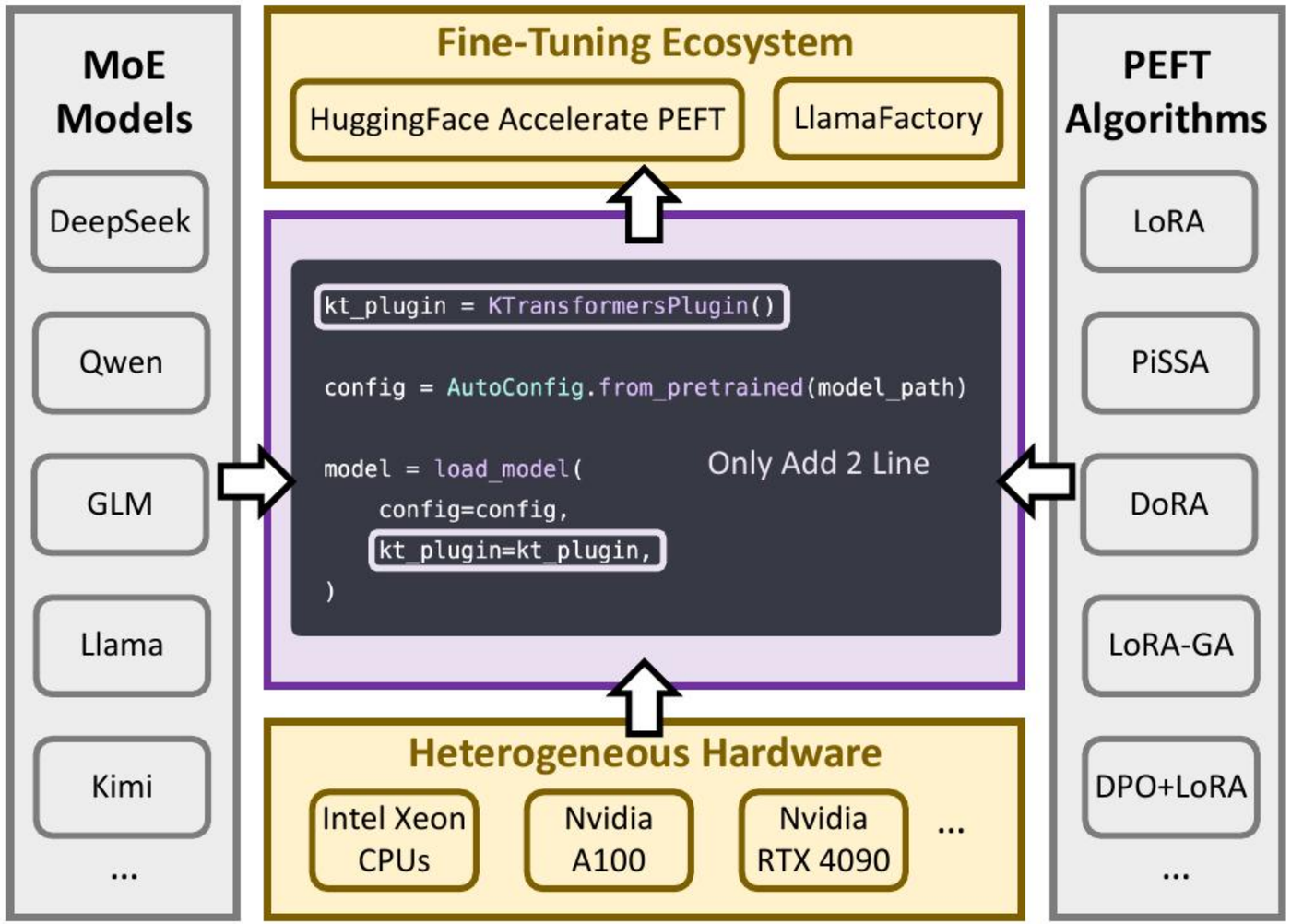
03

Deployment And Ecosystem

Deployment cases, training-inference integration, and the post-train roadmap now reinforce each other.



KT-FT Keeps Original Training Workflows

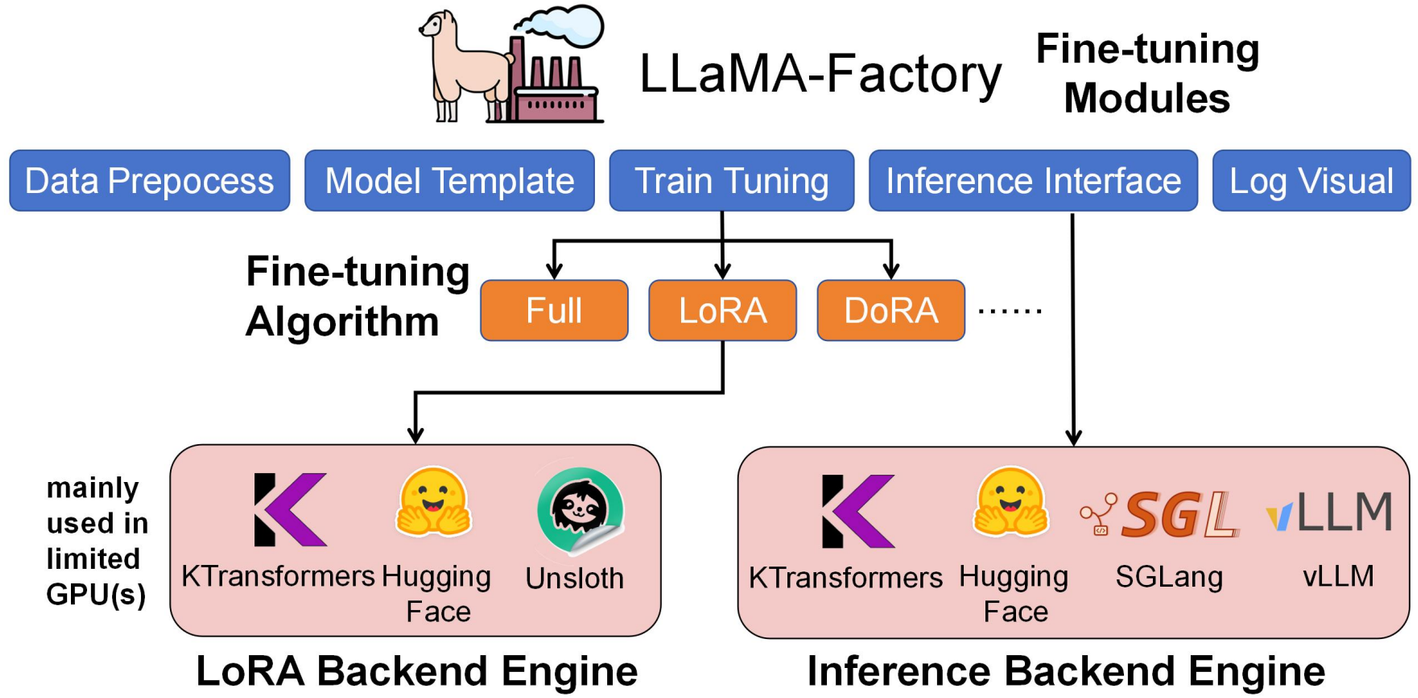


KT-FT changes the backend while users keep the same training setup.

- Hooks into common PEFT workflows.
- Keeps model, algorithm, and hardware boundaries explicit.
- Runtime owns placement, layout, and scheduling decisions.

Original workflows train MoE locally.

KTransformers Connects Fine-Tuning And Inference



One Open-Source Loop

LLaMA-Factory

Fine-Tuning Entry Point

↓

KTransformers Backend

shared heterogeneous runtime for training and inference

↑

SGLang Inference

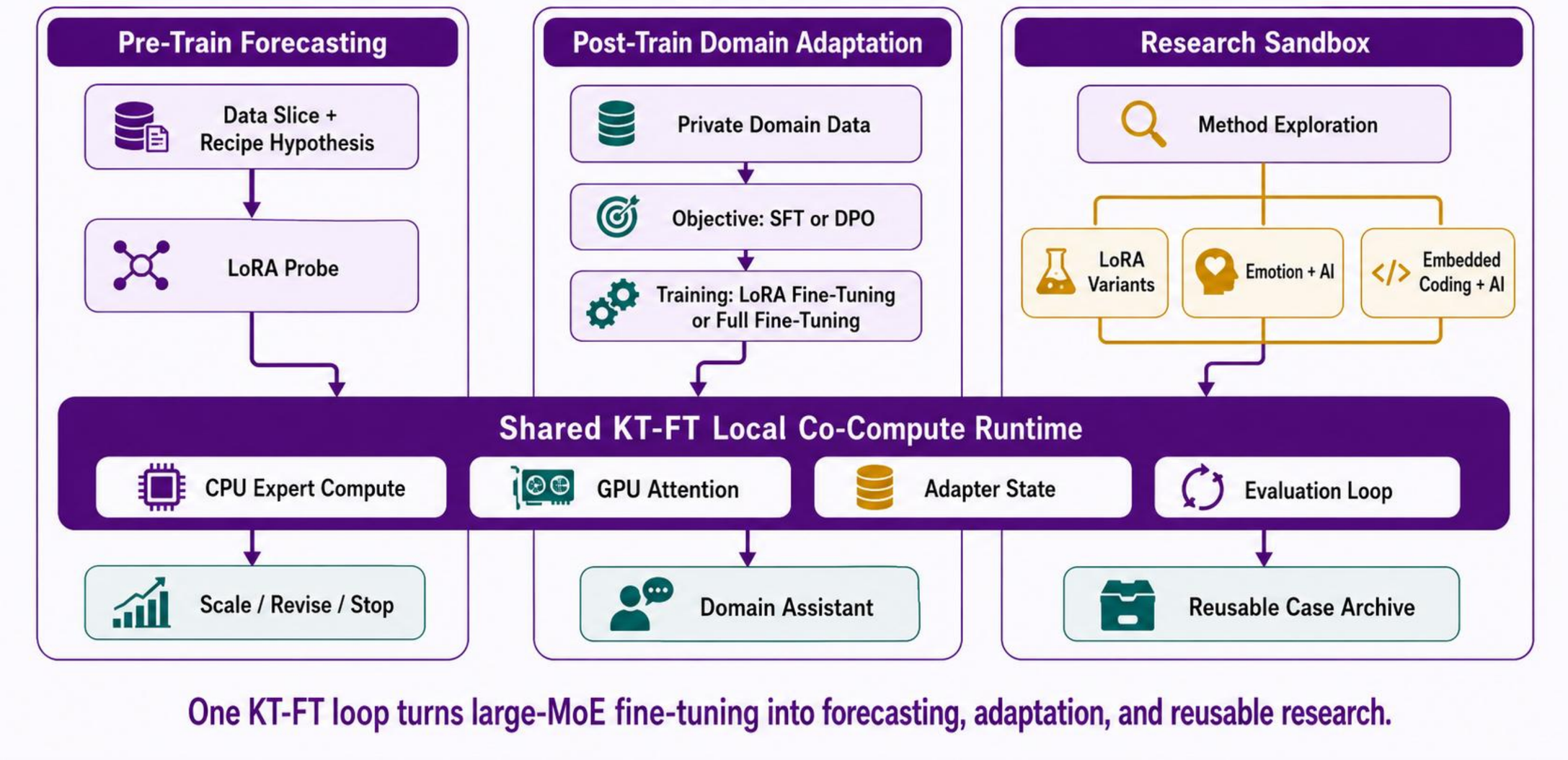
serving path after adaptation

LLaMA-Factory integrates KTransformers as a local fine-tuning backend.

SGLang integration carries the adapted model back to inference.

Serve, tune, evaluate, serve again.

Three Applications Share One KT-FT Loop



Pre-Train: JD.com JoyAI Team

Post-Train: Hisense Group JUHAOKAN AI Technology Center

Research: LoRA Variants And Reusable Cases

Easy to use

Guidance grounded in installation and options docs.

Detailed tutorials

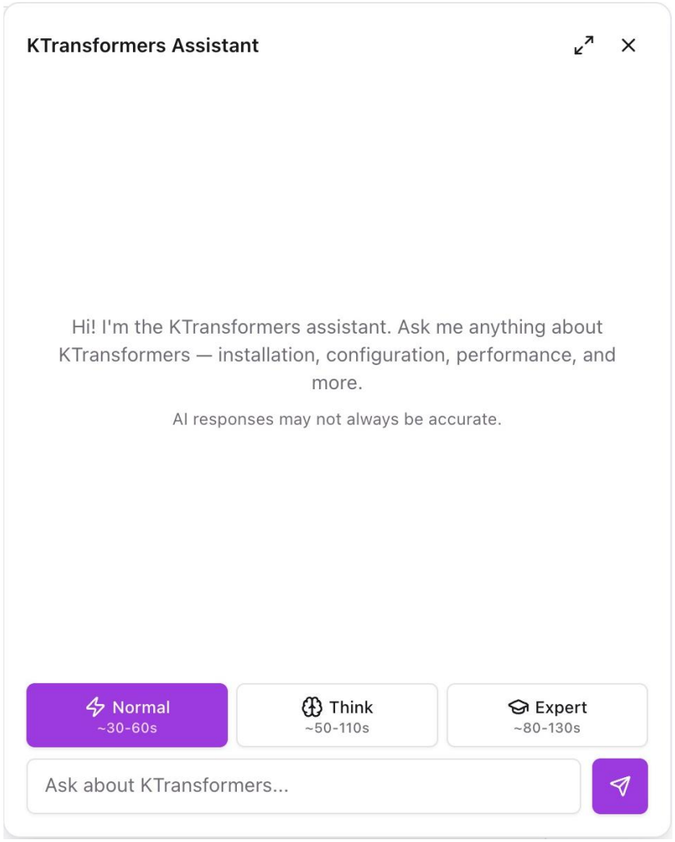
Start Here

Getting Started

- Installation
- DeepSeek V4 Flash Docker Quick Start
- First Inference Server
- First LoRA SFT Run

ktransformers.net

AI setup assistant



Research: Most algorithm papers

GAP

Industry: Frontier-model development

Why GAP: limited compute

1. Small-model tests cannot validate algorithms on large models.

2. Or API / prompt-level work with little local compute.

With KT

CPU memory + co-compute:
less GPU HBM.

Achieve A100-class fine-tuning speed

Explore bigger models.
More practical research. Greater impact.
More industry opportunities!

Join the KTransformers community to connect and discuss.

Scan the WeChat QR code to connect, share cases, and discuss local heterogeneous AI systems.

Discuss	KT-FT, MoE fine-tuning, and system co-design
Join	KTransformers community and open-source roadmap
Build	local inference and fine-tuning workflows

Scan The WeChat QR Code

To Invite Our Community Group Chat

For discussion, collaboration, and KTransformers community updates.

Join The KTransformers Community

GOSIM

GOSIM Shenzhen 2026

04

Four advanced features

Reporter: Yujie Yang

Four advanced features

FINE-TUNING

Feature A	**Native-precision LoRA**	Train LoRA with experts in their original storage format.
Feature B	**Full fine-tuning**	Update base experts, their gradients, and compute layout.
Feature C	**Activation reuse**	Skip a second CPU expert forward during checkpoint replay.

INFERENCE

Feature D	**Layerwise prefill**	Stage expert weights by layer for long prompts.

TRAINABLE LoRA

BF16 compute → FP32 optimizer

FROZEN BASE WEIGHTS

Keep checkpoint-native precision

No full BF16/FP32 copy

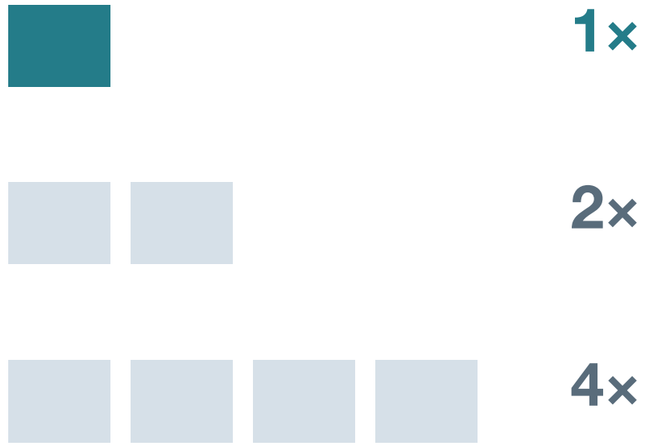
No added quantization loss

BF16	block-FP8	RAWINT4	MXFP4
Qwen MoE	DeepSeek-V3.1	Kimi K2.5	DeepSeek V4

LOW-BIT CHECKPOINT

DeepSeek-V3.1 · FP8

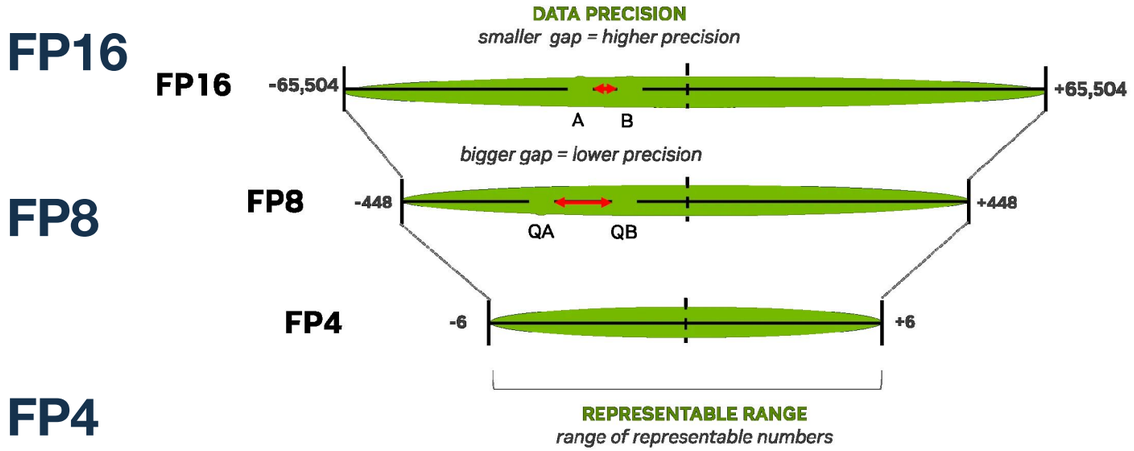
Weight payload



No full BF16/FP32 copy

BF16 CHECKPOINT

Qwen MoE · BF16

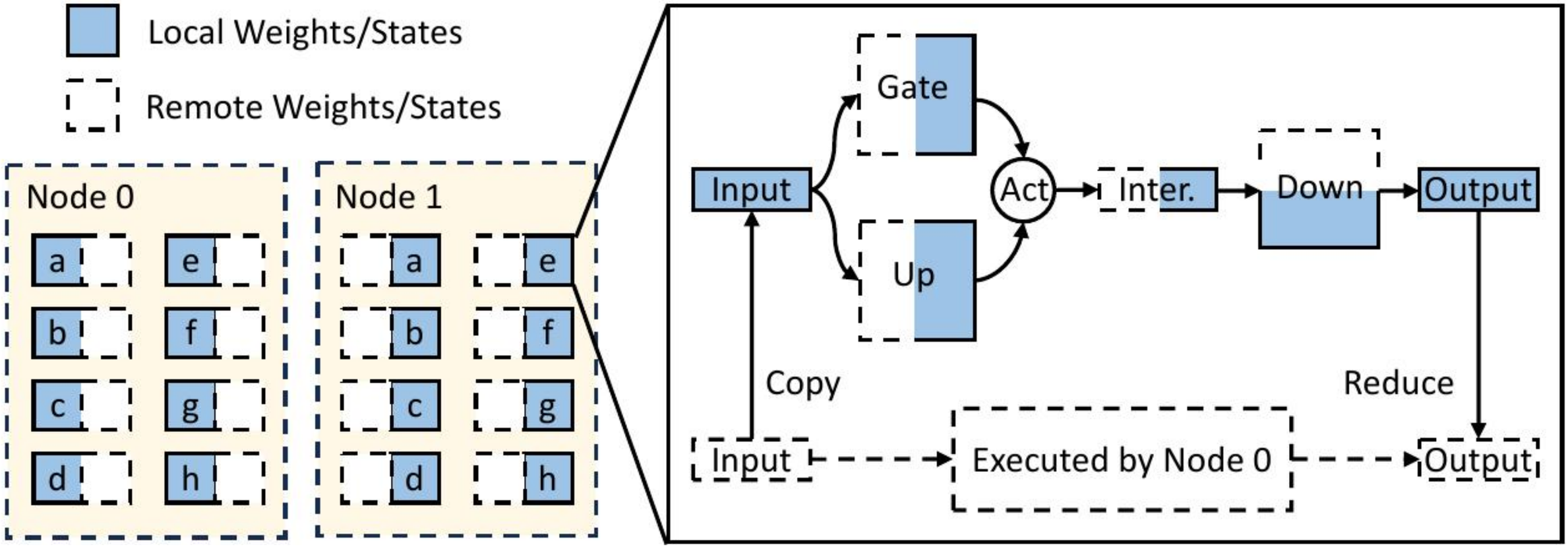


No added quantization loss

Feature B: Full fine-tuning

MODEL	PRECISION
Qwen series	BF16
GLM-4.5-Air	
	To be continued...

Feature B: CPU MoE kernel: NUMA-aware TP



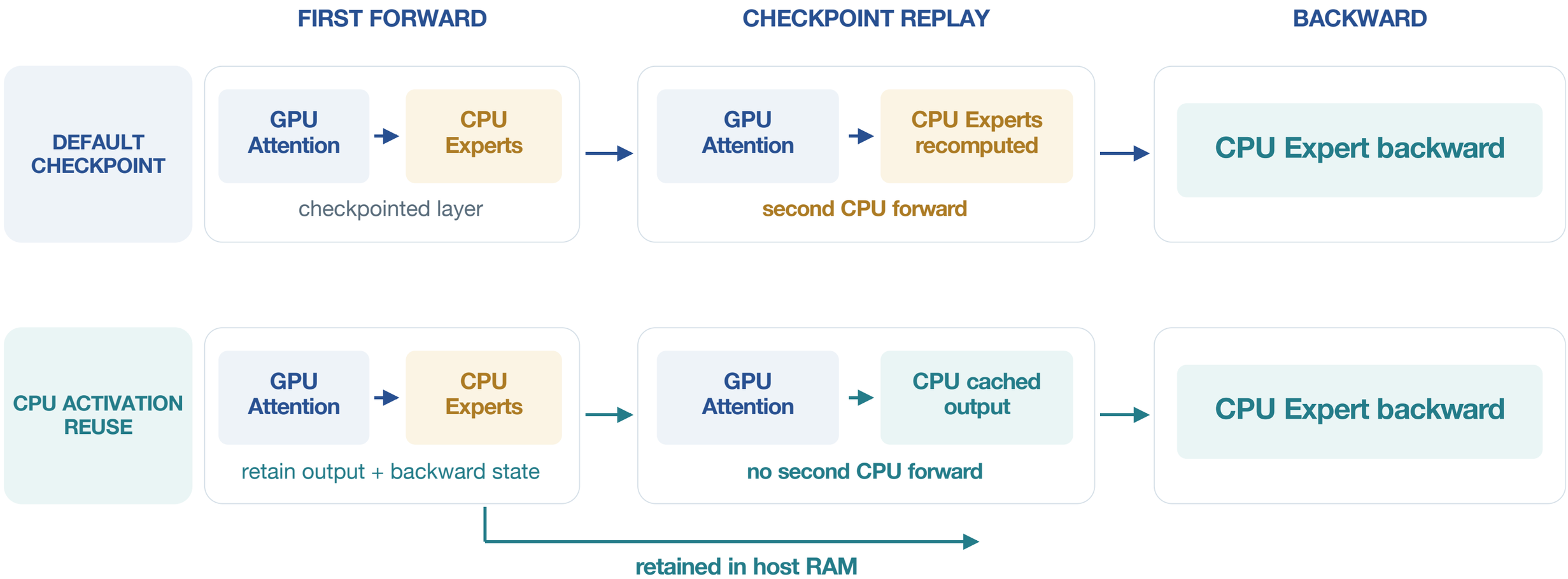
NUMA TP

One shard of every expert per socket

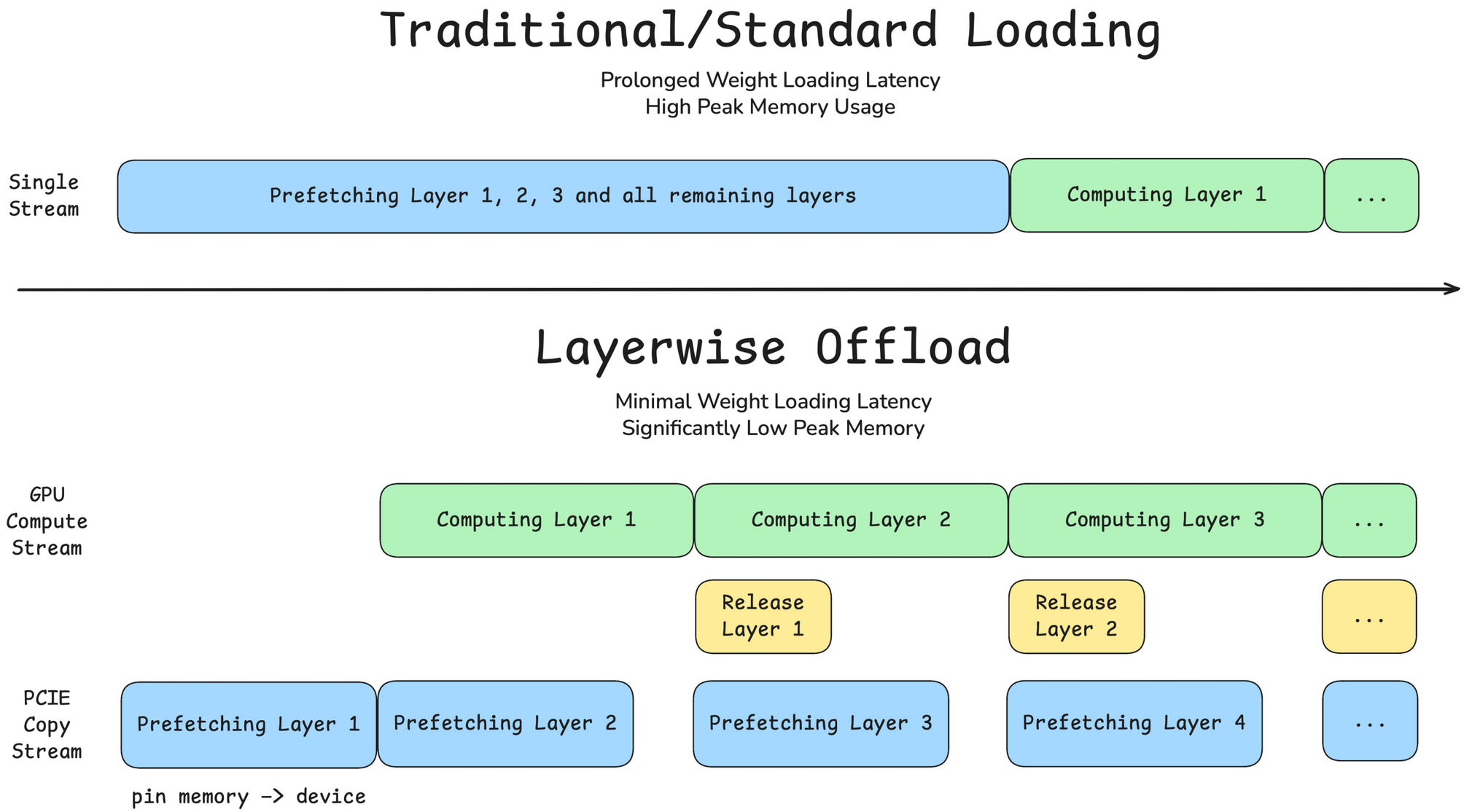
TRAINING EXTENSION

Direct-write optimizer gradients

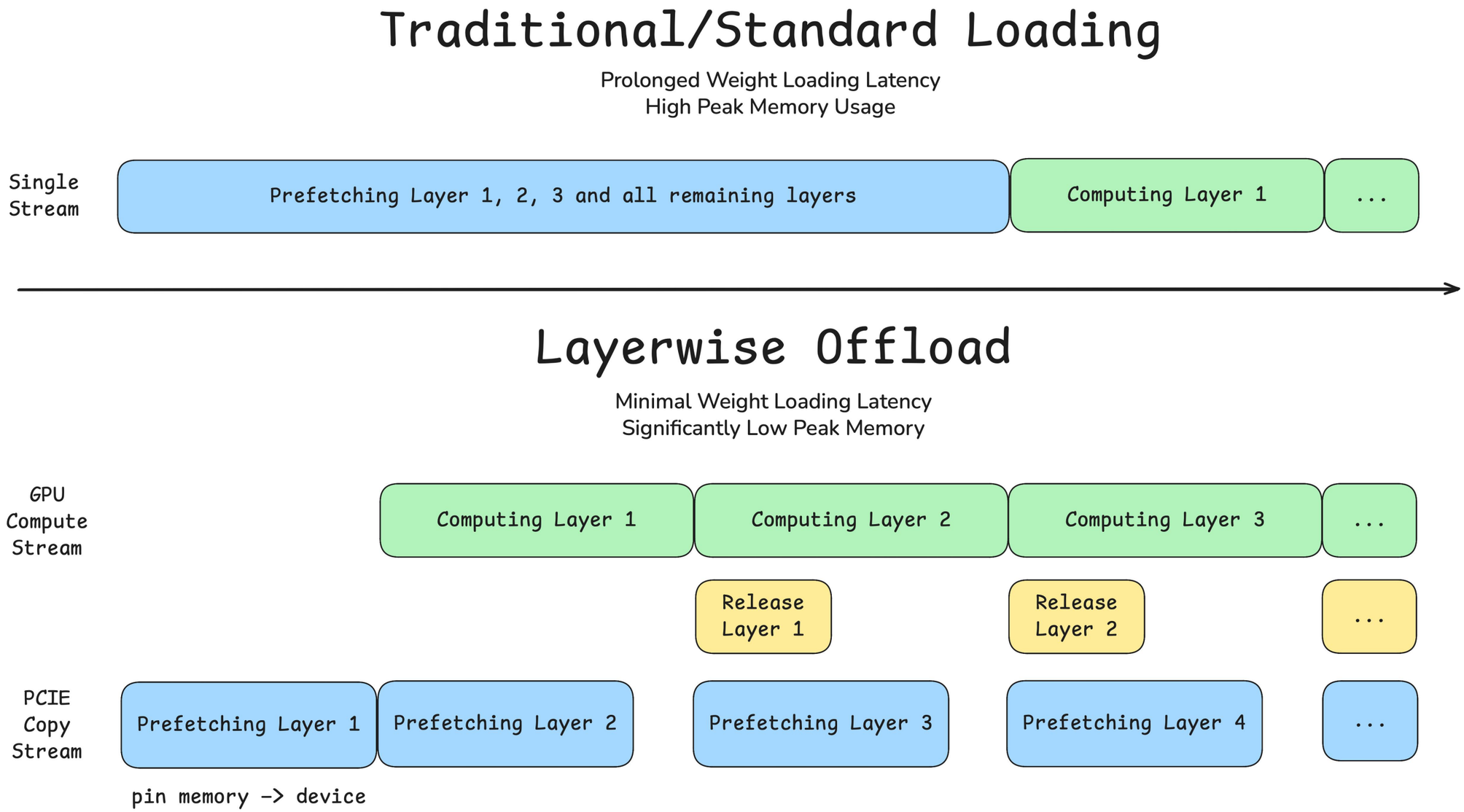
GPU recomputes its layer; CPU experts reuse the first forward



Feature D: Layerwise prefill



Feature D: Layerwise prefill



Feature D: Layerwise prefill

